Header guards и #pragma once

На этом уроке мы рассмотрим, что такое header guards и `#pragma once` в языке C++, а также зачем они нужны и как их правильно использовать.

Оглавление:

1. **Проблема дублирования объявлений**
2. **Header guards**
3. **#pragma once**
4. **Тест**

Проблема дублирования объявлений

Как мы уже знаем из урока о **предварительных объявлениях**, идентификатор может иметь только одно объявление. Таким образом, программа с двумя объявлениями одной переменной получит ошибку компиляции:

```
int main()
{
    int x; // это объявление идентификатора x
    int x; // ошибка компиляции: дублирование объявлений

    return 0;
}
```

То же самое касается и функций:

```
#include <iostream>

int boo()
{
    return 7;
}

int boo() // ошибка компиляции: дублирование определений
{
    return 7;
}

int main()
{
    std::cout << boo();
    return 0;
}
```

Хотя вышеприведенные ошибки легко исправить (достаточно просто удалить дублирование), с **заголовочными файлами** дела обстоят несколько иначе. Довольно легко можно попасть в ситуацию, когда определения одних и тех же заголовочных файлов будут подключаться больше одного раза в файл .cpp. Очень часто это случается при подключении одного заголовочного файла другим.

Рассмотрим следующую программу:

math.h:

```
int getSquareSides()
{
    return 4;
}
```

geometry.h:

```
#include "math.h"
```

main.cpp:

```
#include "math.h"
#include "geometry.h"

int main()
{
    return 0;
}
```

Эта, казалось бы, невинная программа, не скомпилируется! Проблема кроется в определении функции в файле math.h. Давайте детально рассмотрим, что здесь происходит:

→ Сначала main.cpp подключает заголовочный файл math.h, вследствие чего определение функции getSquareSides копируется в main.cpp.

→ Затем main.cpp подключает заголовочный файл geometry.h, который, в свою очередь, подключает math.h.

→ В geometry.h находится копия функции getSquareSides() (из файла math.h), которая уже во второй раз копируется в main.cpp.

Таким образом, после выполнения всех **директив #include**, main.cpp будет выглядеть следующим образом:

```
int getSquareSides()  // из math.h
{
    return 4;
}

int getSquareSides() // из geometry.h
{
    return 4;
}

int main()
{
    return 0;
}
```

Мы получим дублирование определений и ошибку компиляции. Если рассматривать каждый файл по отдельности, то ошибок нет. Однако в main.cpp, который подключает сразу два заголовочных файла с одним и тем же определением функции, мы столкнемся с проблемами. Если для geometry.h нужна функция getSquareSides(), а для main.cpp нужен как geometry.h, так и math.h, то какое же решение?

Header guards

На самом деле решение простое — использовать header guards (защиту подключения в языке C++). **Header guards** — это директивы **условной компиляции**, которые состоят из следующего:

```
#ifndef SOME_UNIQUE_NAME_HERE
#define SOME_UNIQUE_NAME_HERE

// Основная часть кода

#endif
```

Если подключить этот заголовочный файл, то первое, что он сделает — это проверит, был ли ранее определен идентификатор `SOME_UNIQUE_NAME_HERE`. Если мы впервые подключаем этот заголовок, то `SOME_UNIQUE_NAME_HERE` еще не был определен. Следовательно, мы определяем `SOME_UNIQUE_NAME_HERE` (с помощью директивы #define) и выполняется основная часть заголовочного файла. Если же мы раньше подключали этот заголовочный файл, то `SOME_UNIQUE_NAME_HERE` уже был определен. В таком случае, при подключении этого заголовочного файла во второй раз, его содержимое будет проигнорировано.

Все ваши заголовочные файлы должны иметь header guards. `SOME_UNIQUE_NAME_HERE` может быть любым идентификатором, но, как правило, в качестве идентификатора используется имя заголовочного файла с окончанием `_H`. Например, в файле math.h идентификатор будет `MATH_H`:

math.h:

```
#ifndef MATH_H
#define MATH_H

int getSquareSides()
{
    return 4;
}

#endif
```

Даже заголовочные файлы из Стандартной библиотеки C++ используют header guards. Если бы вы взглянули на содержимое заголовочного файла iostream, то вы бы увидели:

```
#ifndef _IOSTREAM_
#define _IOSTREAM_

// основная часть кода

#endif
```

Но сейчас вернемся к нашему примеру с math.h, где мы попытаемся исправить ситуацию с помощью header guards:

math.h:

```
#ifndef MATH_H
#define MATH_H

int getSquareSides()
{
    return 4;
}

#endif
```

geometry.h:

```
#include "math.h"
```

main.cpp:

```
#include "math.h"
#include "geometry.h"

int main()
{
    return 0;
}
```

Теперь, при подключении в main.cpp заголовочного файла math.h, препроцессор увидит, что `MATH_H` не был определен, следовательно, выполнится директива определения `MATH_H` и содержимое math.h скопируется в main.cpp. Затем main.cpp подключает заголовочный файл geometry.h, который, в свою очередь, подключает math.h. Препроцессор видит, что `MATH_H` уже был ранее определен и содержимое geometry.h не будет скопировано в main.cpp.

Вот так можно бороться с дублированием определений с помощью header guards.

#pragma once

Большинство компиляторов поддерживают более простую, альтернативную форму header guards — **директиву #pragma**:

```
#pragma once

// основная часть кода
```

`#pragma once` используется в качестве header guards, но имеет дополнительные преимущества — она короче и менее подвержена ошибкам.

Однако, `#pragma once` не является официальной частью языка C++, и не все компиляторы её поддерживают (хотя большинство современных компиляторов поддерживают).

Я же рекомендую использовать header guards, чтобы сохранить максимальную совместимость вашего кода.

Тест

Добавьте header guards к следующему заголовочному файлу:

add.h:

```
int add(int x, int y);
```

Ответ

```
#ifndef ADD_H
#define ADD_H

int add(int x, int y);

#endif
```